

Annexe : les scripts du TP

Quatre scripts. Deux sont destinés aux étudiants, deux au formateur.

Script	Destinataire	S'exécute sur	Rôle
check-dispo.sh	Étudiants	la VM CLIENT	Sonde de disponibilité. Produit les mesures du procès-verbal.
notify-keepalived.sh	Étudiants	LB1 et LB2	Journalise les transitions VRRP. Preuve horodatée des bascules.
vbox-create-lab.sh	Formateur (ou étudiants)	l'hôte	Crée les 5 VM et câble les 3 réseaux internes.
injecter-panne.sh	Formateur uniquement	l'hôte	Injecte les pannes de la phase 7. Ne pas distribuer.

Tous acceptent `--aide` (ou `-h`) et affichent alors leur propre documentation.

check-dispo.sh : sonde de disponibilité

Interroge une URL à intervalle régulier, journalise chaque requête en CSV, et affiche un bilan à l'arrêt : taux de disponibilité, plus longue interruption (RTO observé), répartition entre les serveurs.

```
# Usage : ./check-dispo.sh [URL] [INTERVALLE_SECONDES] [FICHIER_CSV]

./check-dispo.sh                                # VIP, 1 requête /
200 ms
./check-dispo.sh http://192.168.10.100/ 0.2      # explicite
./check-dispo.sh https://portail.novasante.lan/ 0.5 # après la phase 6
(TLS)
./check-dispo.sh http://192.168.20.21/ 0.2 mesures/web1-direct.csv # un
serveur en direct

# Ctrl+C affiche le bilan.
# Réafficher le bilan d'une mesure passée, sans relancer de sonde :
./check-dispo.sh --bilan mesures/dispo-20260907-141200.csv 0.2
```

Le dernier argument de `--bilan` est l'intervalle utilisé lors de la mesure : il sert à convertir le nombre de requêtes perdues en secondes d'interruption. S'il est faux, le RTO affiché est faux.

Les fichiers CSV sont des livrables. Ils se versionnent tels quels dans `mesures/`, sans retouche.

Le script exige que les serveurs renvoient l'en-tête `X-Served-By` (phase 1.2) : c'est ce qui lui permet d'indiquer quel serveur a répondu. Sans cet en-tête, la colonne serveur affiche `?` et la répartition de charge n'est pas mesurable.

`notify-keepalived.sh` : traçage des bascules VRRP

À déposer en `/etc/keepalived/notify.sh` sur LB1 et LB2, puis à déclarer dans `keepalived.conf` (directive `notify`).

```
sudo install -o root -g root -m 0755 notify-keepalived.sh
/etc/keepalived/notify.sh
sudo mkdir -p /var/log/tp-ha
```

Keepalived l'appelle seul, avec quatre arguments (type, nom, état, priorité). Il écrit dans `/var/log/tp-ha/vrrp-transitions.log` et dans syslog.

Les droits `root:root` en `0755` ne sont pas décoratifs : avec `enable_script_security`, keepalived refuse d'exécuter un script situé dans un emplacement inscriptible par un non-root, et le **désactive silencieusement**.

`vbox-create-lab.sh` : création de la maquette

S'exécute **sur l'hôte**, pas dans une VM. Suppose qu'une VM gabarit nommée `DEBIAN-BASE` existe déjà, éteinte, avec le socle installé et **sans dé-clonage**.

```
./vbox-create-lab.sh                # clone les 5 VM puis câble les
réseaux
./vbox-create-lab.sh --reseau        # recâble les réseaux sans recloner
./vbox-create-lab.sh --demarrer     # démarre les 5 VM, fenêtres
VirtualBox
./vbox-create-lab.sh --demarrer headless # démarre sans fenêtre (SSH
uniquement)
./vbox-create-lab.sh --arreter      # extinction propre des 5 VM
```

```
./vbox-create-lab.sh --destruire      # supprime les 5 VM et leurs disques
./vbox-create-lab.sh --aide          # rappel de l'usage
```

Les deux variables de dimensionnement

Elles se combinent librement et fonctionnent avec tous les modes.

```
PROFIL=mini ./vbox-create-lab.sh      # serveurs à 512 Mo
CLIENT=graphique ./vbox-create-lab.sh # poste client pour un
bureau XFCE
PROFIL=mini CLIENT=graphique ./vbox-create-lab.sh # les deux
CLIENT=graphique ./vbox-create-lab.sh --reseau    # redimensionne sans
recloner
```

PROFIL ne dimensionne que **les quatre serveurs**. CLIENT ne dimensionne que **le poste client** : un client en console n'a aucun besoin de 4 Go, même sur une machine large.

PROFIL	CLIENT	Serveurs	Poste client	Total
mini	console	512 Mo	1 Go, vidéo 16 Mo	3 Go
mini	graphique	512 Mo	2 Go, 2 CPU, vidéo 128 Mo	4 Go
confort (défaut)	console (défaut)	1 Go	1 Go, vidéo 16 Mo	5 Go
confort	graphique	1 Go	4 Go, 2 CPU, vidéo 128 Mo	8 Go

Poste client graphique : ce que le script fait, et ce qu'il ne fait pas

CLIENT=graphique **dimensionne** la VM (RAM, 2 CPU, mémoire vidéo à 128 Mo, contrôleur VMSVGA). Il **n'installe pas** le bureau : cela se fait depuis l'intérieur de la VM, avec apt . Le script rappelle la commande en fin d'exécution.

```
# Dans la VM CLIENT, une fois le réseau configuré
sudo apt install --no-install-recommends xfce4 xfce4-terminal firefox-esr
sudo systemctl set-default graphical.target
sudo reboot
```

Les 128 Mo de mémoire vidéo ne sont pas décoratifs : avec les 16 Mo par défaut, XFCE démarre

mais reste bridé en résolution. Pour un affichage plein écran et le presse-papier partagé, installer en plus les Additions invité VirtualBox.

Choisir entre **gui** et **headless**

`--demarrer` sans argument ouvre une fenêtre par VM. C'est ce qu'il faut au **premier démarrage** : le réseau n'est pas encore configuré, donc SSH n'est pas joignable, et la console VirtualBox est le seul accès.

Une fois l'adressage en place, `--demarrer headless` démarre les cinq VM sans aucune fenêtre et l'on administre entièrement en SSH. C'est plus léger et plus proche de la réalité d'un parc serveur. Un poste client graphique reste évidemment à lancer en `gui`.

Les VM démarrent dans l'ordre serveurs puis client, avec deux secondes d'écart, pour ne pas saturer l'hôte par cinq démarrages simultanés.

`--reseau` est le mode le plus utile en séance : il réapplique les trois réseaux internes et le **mode promiscuité `allow-all`**, qui est la cause numéro un des clusters VRRP qui ne se voient pas. Si un binôme a deux MASTER dès le départ, c'est la première chose à repasser.

`--detruire` demande de saisir `SUPPRIMER` en toutes lettres avant d'agir.

Le script **ne fait pas** le dé-clonage interne (nom d'hôte, `machine-id`, clés d'hôte SSH) : c'est le travail de la phase 0.3, et c'est un apprentissage à part entière.

`injecter-panne.sh` : pannes de la phase 7 (formateur)

S'exécute **sur l'hôte**. Injecte une panne, l'horodate dans un journal, et n'affiche rien sur l'écran de l'étudiant.

```
./injecter-panne.sh liste           # les 9 pannes, avec le comportement
attendu de chacune
./injecter-panne.sh 4               # injecte la panne n° 4 (disque plein sur
WEB1)
./injecter-panne.sh restaurer       # remet la maquette en état
./injecter-panne.sh journal         # relit toutes les injections horodatées
```

Plusieurs maquettes sur un même hôte : passer un suffixe de nom de VM en second argument.

```
./injecter-panne.sh 1 -groupe3     # agit sur LB1-groupe3, WEB1-groupe3,
etc.
./injecter-panne.sh restaurer -groupe3
```

Le journal est écrit dans `~/tp-ha-injections.log`, modifiable par la variable `JOURNAL_PANNES`. **Comparer l'heure d'injection à l'heure de détection déclarée dans le procès-verbal du binôme est en soi un critère de notation.**

Les pannes 2, 4, 6 et 7 passent par SSH vers les VM et supposent donc un compte `admin` joignable sur le réseau `HA-SYNC`. Les pannes 1, 3, 5, 8 et 9 utilisent `VBoxManage` et fonctionnent quoi qu'il arrive, même si les étudiants ont durci SSH.

Injection manuelle, sans le script

Utile pour les tests des étudiants eux-mêmes en phases 2 à 6.

```
VBoxManage controlvm LB1 poweroff          # panne matérielle brutale
VBoxManage controlvm LB1 pause              # machine gelée, ni éteinte ni
répondante
VBoxManage controlvm LB1 resume             # la réveiller
VBoxManage controlvm LB1 setlinkstate2 off  # débranche le câble de
l'adaptateur 2
VBoxManage controlvm LB1 setlinkstate2 on   # le rebranche
```

Le numéro passé à `setlinkstate` est celui de **l'adaptateur VirtualBox**, pas celui de l'interface Linux : adaptateur 2 correspond en général à `enp0s8`. Vérifier avec `ip -br link` avant de conclure quoi que ce soit d'un test.